unidb — Engine Architecture & Product Guide

One embedded database engine that unifies relational tables, vector search, graph relationships, and a durable event stream over a single storage core — so one transaction can save a row, its embedding, a graph edge, and an event **atomically**, in one commit.

> **The one idea to remember.** A typical AI-native app stitches together Postgres + a vector store + a graph database + a message queue, and writes to all four on every change with no shared transaction. unidb collapses that into **one engine and one commit**. Measured against exactly that four-system stack at matched flush-to-disk durability, unidb is **3.61× faster** and loses **zero** data on a crash where the four-system stack tears a record.

Contents

1. What unidb is

unidb is a single embedded database engine written in Rust. Like SQLite, it runs **inside your application** as a library — no separate server process required — and keeps all of your data in one file. Unlike SQLite, it natively handles four kinds of data at once, over one shared storage and transaction core:

- **Relational tables** — a practical SQL dialect over versioned tables: joins, aggregates, subqueries, CTEs, a cost-based query optimizer, and `EXPLAIN [ANALYZE]`.
- **Vector search** — `VECTOR(n)` columns with a durable on-disk similarity index and a `NEAR` operator for nearest-neighbor queries (the building block of semantic / AI search).
- **Graph relationships** — directed edges between entities, with a Cypher-style read query for traversals.
- **Durable event stream** — a built-in, Kafka-style event queue with consumer offsets and replay, for change-data feeds and realtime.

Because all four live in **one engine, one log, and one transaction manager**, a single transaction can touch all of them and either commit everything or nothing. That is the capability an application otherwise has to fake with distributed writes across separate systems — and can never make truly atomic.

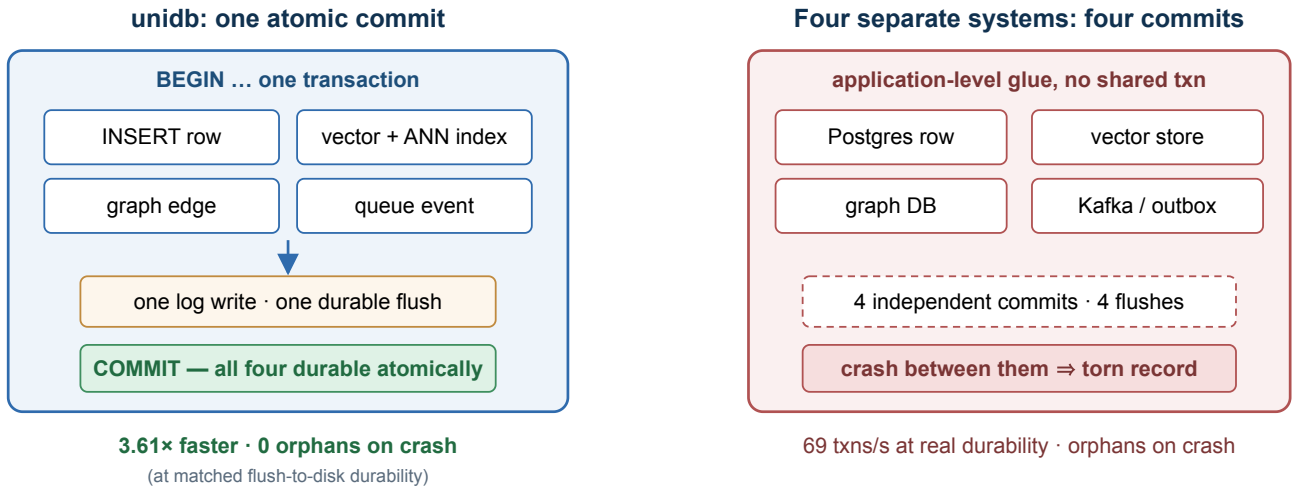


Figure 1 — One atomic multi-model commit vs. the four-system stack it replaces.

> **What unidb is not.** It is honest about its scope. It is not trying to beat Postgres on every single-table workload, it does not implement the entire ANSI SQL standard, and it runs as a single primary with read replicas (no distributed multi-writer consensus). Its advantage is the unified, multi-model, single-commit workload — the exact pattern AI-native apps are built on.

2. Architecture at a glance

2.1 The layered engine

Every capability is built as a thin layer on top of the same storage core. That is what makes cross-model atomicity *structural* rather than a promise: a graph edge and a queue event are, underneath, just ordinary rows in the same store, protected by the same log.

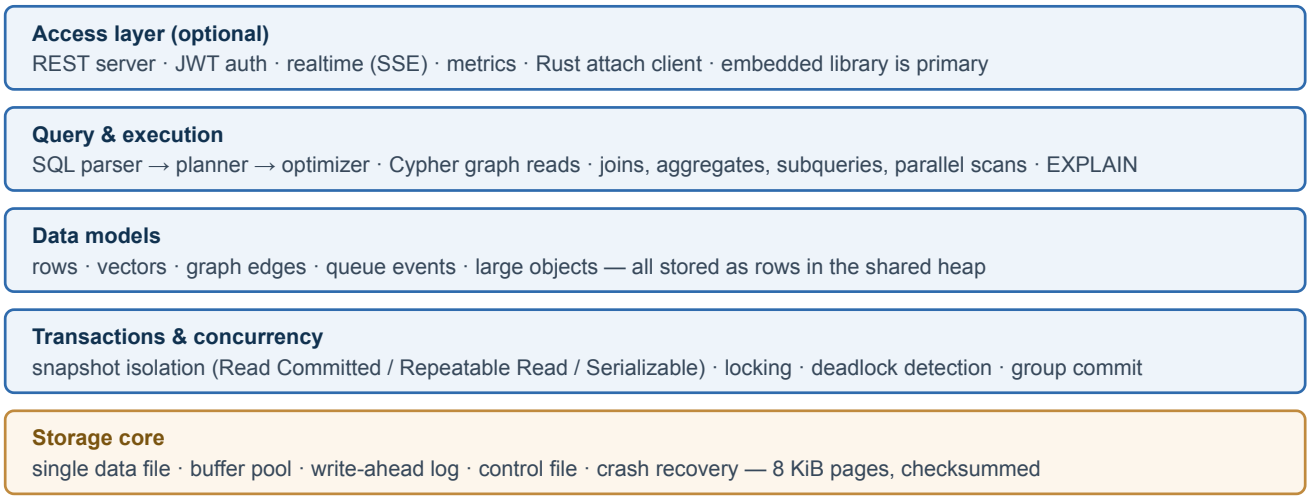


Figure 2 — The layered engine. Everything sits on one shared storage core.

2.2 Core guarantees

- **Fully durable (ACID).** Every committed transaction is on disk before the commit returns. A committed change survives a crash; an in-flight one leaves no trace.
- **Runs in your process, thread-safe.** The engine is shared safely across threads; many readers and writers work concurrently and scale with CPU cores. The default embedded build pulls in **no async runtime and no network stack** — it is a plain, synchronous library.
- **Log-before-data, always.** No change ever reaches the data file before it is safely recorded in the write-ahead log. This single rule is what makes crash recovery provably correct.
- **Explicit transactions.** Every operation runs inside a transaction you begin and then commit or roll back — no surprising implicit commits.

2.3 Ways to run it

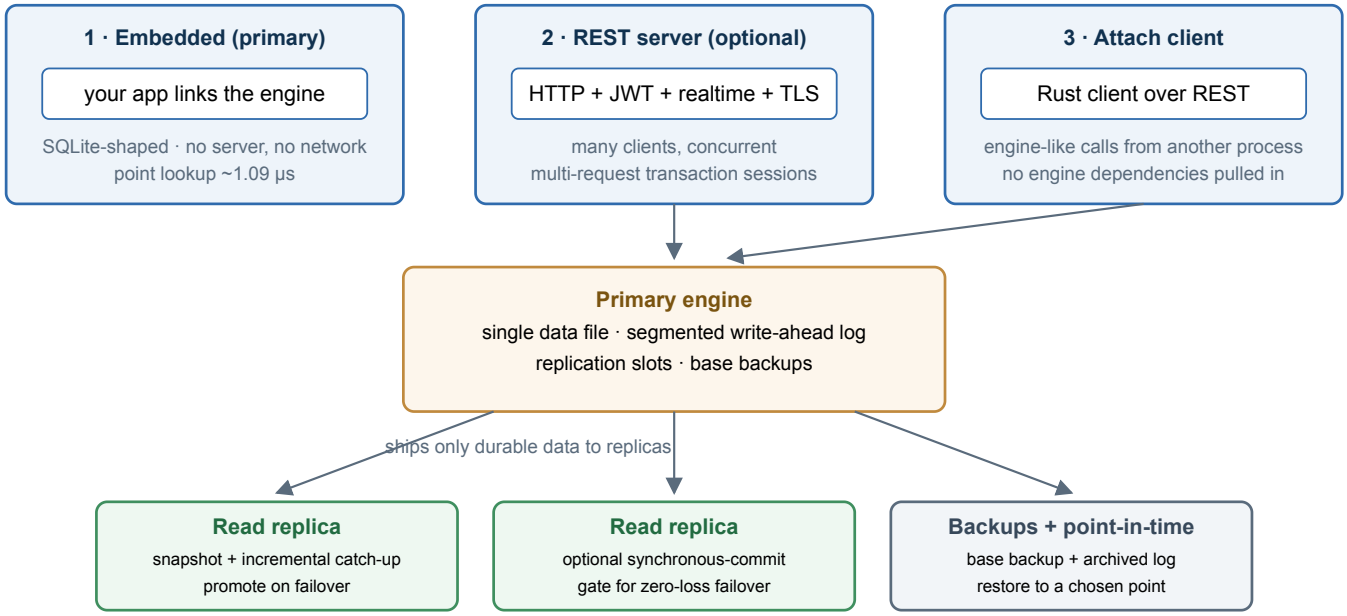


Figure 3 — Deployment modes: embedded library (primary), optional REST server, Rust attach client, plus single-primary + read-replica high availability.

3. How your data is stored & kept safe

3.1 On-disk format

- **One data file** made of fixed-size **8 KiB pages**, plus a separate, segmented write-ahead log directory. Keeping data in one file keeps backup, copy, and recovery simple.
- **Every page is checksummed** and carries a version stamp, so corruption is detected on read rather than silently returned.
- **Rows are stored in slotted pages.** Each row carries a small header that records which transaction created it and which (if any) superseded it — the basis of unidb's versioning (Section 4). New column types are added without ever changing the on-disk format.

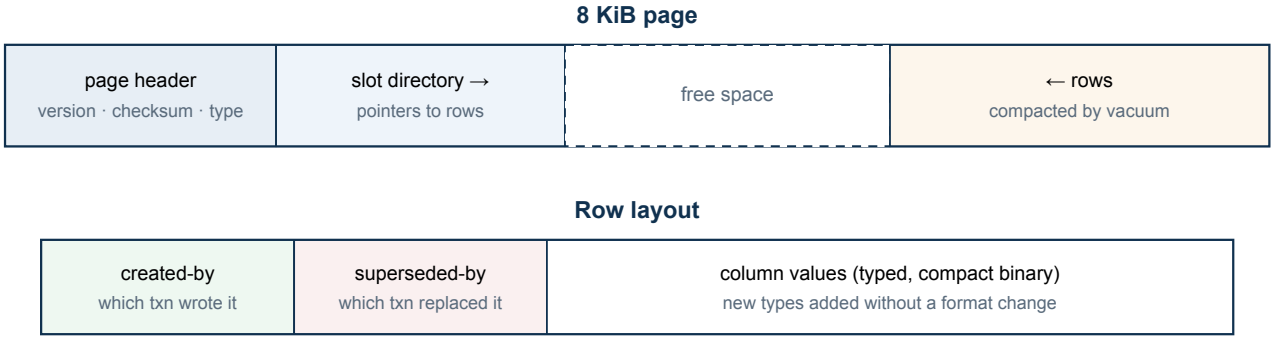


Figure 4 — Page and row layout. Every page is checksummed; every row records its version stamps.

What you'll actually see on disk

Point unidb at a directory and this is what appears inside it. Everything here is plain files and folders — nothing exotic, nothing that needs a special tool to inspect at the filesystem level.

Name	Type	What it holds	How it grows
`control`	File	A small pointer telling recovery exactly where to start — the page size in use, where the schema lives, and the last safe checkpoint.	Fixed size; rewritten in place at every checkpoint, never grows.
`data.db`	File	The database itself — every table's rows, every index, and the schema, as the 8 KiB pages described above.	Grows in small fixed-size chunks only as new space is actually needed. No size ceiling.
`db.wal/`	Folder	The write-ahead log — every change, recorded here before it's ever applied to `data.db` (Section 3.2). Contains one or more numbered log-segment files.	Each segment file is capped at a fixed size; once full, a new segment starts. Old segments are deleted automatically once a checkpoint confirms they're no longer needed for recovery.
`timeline.bin`	File	A small marker appended for every committed transaction, used only to resolve a "restore to this point in time" request to the right log position (Section 9). Absent if you've never taken a backup.	Grows by a few bytes per commit; no automatic cleanup yet, so a very long-running, high-commit-rate instance is worth keeping an eye on.

`audit.log`	File	Security-relevant events — user, role, and permission changes — appended as plain text. Empty until your first such change.	Append-only; grows with auth activity, not with ordinary data traffic.
`logs/`	Folder	*REST server only* — the server's own operational log, not something an embedded-only use creates.	One file per calendar day, rotated automatically at midnight.

If you find some other file sitting in the same directory — a stray `server.log` or similar — it likely isn't something unidb wrote there itself; it's usually just wherever a shell redirected that process's console output when it was started. Safe to move or delete independently of the files above.

3.2 How a write becomes permanent

unidb uses a write-ahead log (WAL): before a change is applied to a page, a record describing it is appended to the log. The log is the source of truth. A commit is not acknowledged until its log record is physically flushed to disk.

Within a transaction, individual statements append to the log *without* flushing; the single flush happens at `COMMIT`. When many transactions commit at once, the engine **coalesces them behind one shared flush** ("group commit") — so throughput scales with concurrency instead of being bottlenecked by one flush per statement. Read-only transactions skip the flush entirely.

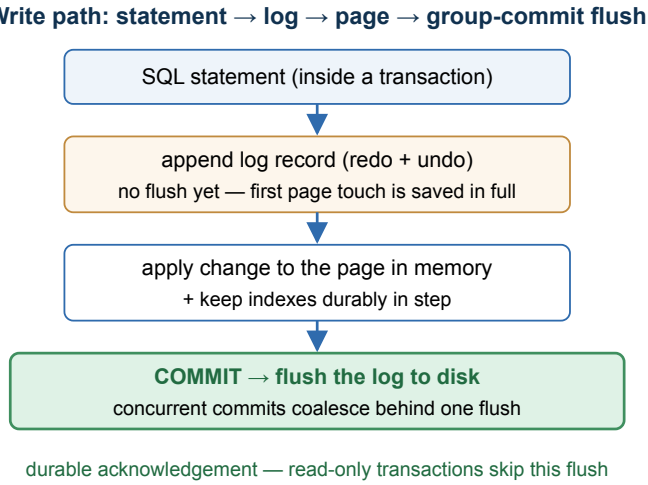


Figure 5 — The write path. Log-before-page is enforced at every change; durability is one shared flush per commit.

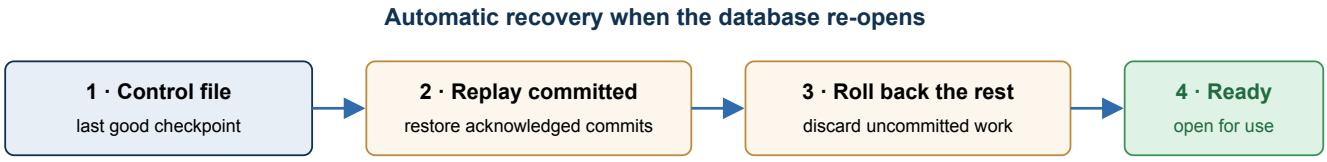
3.3 Crash recovery — when the power goes out

If the process is killed or the machine loses power mid-write, you do nothing special: the next time the database is opened, unidb **repairs itself automatically**. Recovery is deterministic and needs no manual intervention.

On open, the engine:

1. Reads the **control file** — a tiny, checksummed record that always points at the last known-good checkpoint. Recovery always starts here.
2. **Replays** committed changes from the log, forward from that checkpoint, so every acknowledged commit is restored.
3. **Rolls back** anything belonging to transactions that never reached a commit, leaving no partial rows behind.

> **The guarantee.** After recovery, the database contains **exactly the transactions that had committed** — no more, no less. No half-applied statements, no orphaned rows, no readable corruption. This holds even if the crash happens *during* recovery itself: re-running recovery is safe and produces the same result.



Even total loss of the data file can be rebuilt from the log alone; a checkpoint periodically trims the log so recovery stays fast.

Figure 6 — Crash recovery on open: control file → replay committed → roll back uncommitted.

3.4 Protection against corruption

- **Torn-page protection.** An 8 KiB page write is not atomic at the hardware level — a crash mid-write can leave a page half-old and half-new. The first time a page changes after each checkpoint, unidb saves a full, clean image of it to the log, so recovery can always rebuild a torn page from a known-good base. This closes the single most common silent data-loss hole in naive storage engines.
- **Honest handling of disk-flush failures.** If the operating system ever reports that a flush failed, unidb refuses to keep claiming durability and requires a restart, rather than falsely reporting data as safe. After the restart, recovery still finds a consistent database.
- **Checksums on every page** catch bit-rot and partial writes on read.

3.5 Memory & space management

- **Buffer pool.** Hot pages are cached in a configurable in-memory pool (32 MiB by default). Working sets larger than the pool are handled safely — the engine flushes and evicts correctly rather than failing.
- **Free-space tracking.** Each table tracks its free space durably, so inserts stay fast and predictable even as tables grow into the millions of rows — insert cost stays flat where a naive design would degrade and eventually fail.
- **Vacuum & autovacuum.** Because updates create new row versions, superseded versions are reclaimed in the background once no transaction can still see them, keeping tables compact automatically (see Section 9).
- **No practical schema-size limit.** Your table and column definitions are stored durably and can spill across as many pages as needed, so wide schemas — many tables, many columns, heavy use of statistics — grow without hitting an artificial ceiling.

4. Transactions & working with many users at once

4.1 Snapshots & isolation levels

unidb uses **multi-version concurrency control (MVCC)**: an update never overwrites a row in place — it writes a new version and marks the old one as superseded. Each transaction reads from a consistent **snapshot**, so **readers never block writers and writers never block readers**. Old versions are cleaned up later, once no active transaction can still see them.

Three isolation levels are available:

Level	What you get	On conflict
Read Committed (default)	Each statement sees the latest committed data. The common, high-throughput default.	A write against a row another transaction is still changing is rejected so you can retry.
Repeatable Read	The whole transaction sees one stable snapshot — the same rows every time you read them.	Conflicting concurrent writes surface a serialization error; the caller retries.
Serializable	The strongest guarantee: the result is as if transactions ran one at a time, protecting against subtle write-skew anomalies.	A dangerous interleaving is detected and one transaction is asked to retry.

4.2 Reading and writing at the same time

unidb is designed for real concurrency, and its behavior under load is straightforward to reason about:

- **Reads are lock-free.** Any number of readers proceed in parallel and never wait on each other or on writers.
- **Writers to different rows run in parallel** and scale across CPU cores. In benchmarks, raw write throughput scales **3.68× at 8 concurrent writers**, and concurrent inserts into indexed tables run about **25% faster** than the earlier serialized behavior.
- **Writers to the same row are safe by first-committer-wins.** If two transactions try to change the same row, one succeeds and the other gets a clear conflict error to retry — there is no lost update and no silent overwrite.
- **Deadlocks are detected automatically.** If two transactions wait on each other, the engine breaks the cycle by aborting one with a deadlock error, rather than hanging.
- **Per-query safety limits.** Queries can be given time budgets and can be cancelled, so one runaway query cannot monopolize the engine.

Under the hood, concurrent index updates are kept correct with fine-grained, carefully ordered locking so that inserts can proceed in parallel without ever producing an inconsistent index — validated by an extensive concurrency test suite (Section 12).

4.3 What happens on rollback

When a transaction is rolled back (explicitly, or because it hit an error or was abandoned), unidb physically neutralizes everything it wrote: rows it inserted become invisible and rows it had marked superseded are restored. This is what lets graph edges, events, and large objects all inherit correct rollback behavior for free — they are ordinary rows, so undoing them uses the exact same machinery as undoing a table row.

5. The SQL layer

unidb speaks a practical, PostgreSQL-flavored SQL dialect. Statements are parsed, planned, cost-optimized, and executed against versioned tables. You can inspect any plan with `EXPLAIN` (and its real row counts and timing with `EXPLAIN ANALYZE`).

5.1 Supported data types

SQL type	Stores	Notes
`INT` / `INTEGER` / `BIGINT`	64-bit signed integer	—
`SERIAL`	Auto-incrementing integer	Durable per-table counter; typical primary-key type.
`FLOAT` / `DOUBLE`	64-bit floating point	—
`DECIMAL(p,s)` / `NUMERIC(p,s)`	Exact fixed-point decimal	No precision loss — ideal for money.
`TEXT`	UTF-8 string	Can carry a full-text index.
`BOOL` / `BOOLEAN`	true / false	—
`JSON`	Nested JSON document	Returned as real nested JSON, not a string.
`UUID`	128-bit UUID	—
`DATE`	Calendar date	—
`TIME`	Time of day	—
`TIMESTAMP`	Date + time (UTC)	Serialized as a string so no precision is lost.
`BYTEA`	Raw binary bytes	—
`VECTOR(n)`	Fixed-length numeric vector	Embeddings for similarity search; can carry a vector index.

New types are added additively, without ever changing the on-disk format — existing databases keep working across engine upgrades.

5.2 Writing queries — examples

Create a table

```
CREATE TABLE users (
  id        SERIAL PRIMARY KEY,
  email     TEXT NOT NULL UNIQUE,
  full_name TEXT,
  plan      TEXT DEFAULT 'free',
  credits   DECIMAL(10,2) DEFAULT 0.00,
  is_active BOOL DEFAULT true,
  created   TIMESTAMP,
  CHECK (credits >= 0)
);

CREATE TABLE documents (
  id        SERIAL PRIMARY KEY,
  owner_id  INT REFERENCES users (id),
  title     TEXT,
  body      TEXT,
  embedding VECTOR(1536)
);
```

Insert data (use bind parameters to stay injection-safe)

```
INSERT INTO users (email, full_name, plan)
VALUES ($1, $2, $3);
-- params: ["ada@example.com", "Ada Lovelace", "pro"]

INSERT INTO documents (owner_id, title, body, embedding)
VALUES (1, 'Notes', 'first draft', [0.12, 0.03, ...]);
```

Query — filters, joins, aggregates, sorting

```
-- Join + filter
SELECT u.email, d.title
FROM documents d
JOIN users u ON u.id = d.owner_id
WHERE u.plan = 'pro'
ORDER BY d.title
LIMIT 20;

-- Aggregate with grouping
SELECT plan, COUNT(*) AS n, AVG(credits) AS avg_credits
FROM users
WHERE is_active = true
GROUP BY plan
HAVING COUNT(*) > 5;

-- Subquery
SELECT email FROM users
WHERE id IN (SELECT owner_id FROM documents WHERE title LIKE 'Notes%');

-- Common table expression (CTE)
WITH active AS (SELECT id, email FROM users WHERE is_active = true)
SELECT * FROM active ORDER BY email;
```

Vector similarity search

```
-- Ten documents whose embedding is nearest to the query vector
SELECT id, title
FROM documents
WHERE NEAR(embedding, [0.10, 0.24, ...], 10);
```

`NEAR(column, query_vector, k)` returns the `k` nearest rows by vector distance and composes with ordinary `WHERE` filters and security policies.

Inspect a query plan

```
EXPLAIN ANALYZE
SELECT plan, COUNT(*) FROM users GROUP BY plan;
```

Traverse graph relationships (Cypher)

```
MATCH (a)-[:FOLLOWS]->(b)
WHERE a.id = 1
RETURN b.id;
```

5.3 Query capabilities

Area	Supported	Not yet supported
Joins	INNER / LEFT / RIGHT / CROSS, `JOIN … USING (cols)` ; hash join (spills to disk on large inputs), sort-merge, and index-nested-loop	FULL OUTER, NATURAL

Aggregation & sort	COUNT / SUM / AVG / MIN / MAX, GROUP BY / HAVING, DISTINCT, ORDER BY (spills to disk), LIMIT / OFFSET	Window functions
Subqueries & CTEs	Scalar / IN / EXISTS (correlated and not), IN-list, non-recursive `WITH`	Recursive CTEs
Planning	ANALYZE (durable statistics: row counts, distinct counts, histograms), cost-based join ordering, index-vs-scan choice, EXPLAIN [ANALYZE]	—
Schema (DDL)	CREATE / ALTER / DROP / TRUNCATE, DROP COLUMN, SERIAL, prepared statements with `$n` parameters	Views, triggers, stored procedures

Join, aggregate, and subquery results are continuously cross-checked against SQLite on shared data to guarantee correctness.

5.4 Constraints, security & roles

- **Constraints:** `PRIMARY KEY`, `FOREIGN KEY` / `REFERENCES`, `UNIQUE`, `NOT NULL`, `CHECK`, and `DEFAULT` — enforced on every insert and update.
- **Row-level security (RLS):** attach a policy predicate to a table (e.g. `tenant_id = 7`) and it is automatically applied to every query on that table — so multi-tenant isolation is enforced by the engine, not by every query author. It composes with vector search and index paths alike.
- **Users, roles & privileges:** create users and roles, grant per-table privileges (with role inheritance), and run statements as a given user so those privileges are enforced. Over the REST server, the caller's identity comes from their signed token.
- **Injection-safe by construction:** prepared statements with `$n` bind parameters pass values as data, never as re-parsed SQL.

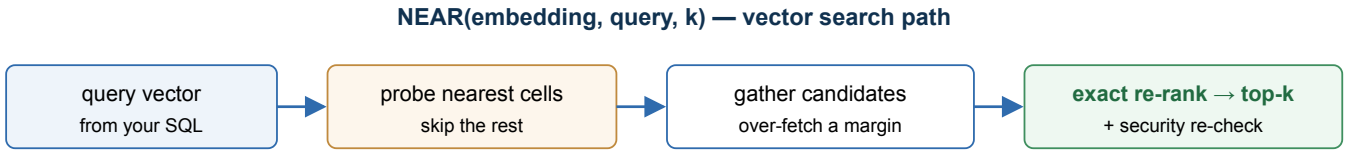
6. Search, indexing & relationships

6.1 Automatic durable indexes

Every secondary index in unidb — B-tree, vector, full-text, and graph adjacency — is **durable, crash-safe, and kept in step with your data inside the same transaction**. Indexes are never lost or rebuilt on open, so opening a database is instant no matter how large it is. Index-assisted queries always re-check results against your transaction's snapshot and security policy, so an index can only ever make a query *faster*, never return a wrong or unauthorized row.

6.2 Vector similarity search

A `VECTOR(n)` column can be given a durable similarity index, queried with the `NEAR` operator. unidb clusters vectors into cells, probes the most promising cells for a query, then re-ranks the candidates exactly — so there is **no approximation error in the final ranking**. In testing it returns the same top results as a brute-force search (recall@10 = 1.000) while opening instantly and surviving crashes with its results intact.



Distance metrics: Euclidean (default) and cosine. The index is built once and kept durable — never recomputed on open.

Figure 7 — The `NEAR` vector-search path: probe promising cells, then re-rank exactly.

Embedding search over stored files

unidb does not read file formats or generate embeddings itself — turning a stored PDF, document, or image into a vector is an application step, the same as anywhere else vector search is used. What unidb removes is the plumbing around it: because object metadata already lives in ordinary tables, adding embedding search is a plain schema addition, not a new subsystem.

```
-- One extra column alongside your file metadata
CREATE TABLE file_embeddings (
  bucket      TEXT,
  object_key  TEXT,
  embedding   VECTOR(1536)
);

-- After your app extracts the file's text and calls an embedding model:
INSERT INTO file_embeddings (bucket, object_key, embedding)
VALUES ('docs', 'reports/q3.pdf', [0.02, 0.44, ...]);

-- Search by meaning, then join back to get the file
SELECT o.bucket, o.object_key, o.content_type
FROM   file_embeddings e
JOIN   objects o ON o.bucket = e.bucket AND o.object_key = e.object_key
WHERE  NEAR(e.embedding, [0.03, 0.41, ...], 10);
```

For a small file stored inline, the embedding insert can share the same transaction as the upload's metadata write, so "save the file + its embedding" is one atomic commit. For a large file tiered to external storage, the bytes move by a direct presigned upload outside the engine transaction — tie the embedding write to your upload-confirmation step instead. If what you need is exact keyword matching over extracted text rather than similarity search, index it as full text instead (below) — the same store-then-index pattern applies either way.

6.3 Full-text search

A `TEXT` column can carry a durable full-text (inverted) index. Search tokenizes your query and intersects the matching documents, then checks each candidate against your snapshot and security policy. Raw lookups run in the tens of microseconds.

6.4 Graph relationships

- **Edges are first-class but cost nothing extra to make safe.** A directed edge `(from, to, type, properties)` is stored as an ordinary row, so per-edge locking, rollback, and crash safety come for free.
- **Fast neighbor lookups.** A durable adjacency index finds a vertex's edges, and unidb batches the row fetches by page so a hot, highly-connected vertex costs one page read per ~128 edges instead of one per edge — about **9× faster** than the naive approach, bringing graph traversal to parity with Postgres.
- **Cypher reads:** `MATCH (a)-[:TYPE]->(b) WHERE … RETURN …` for single-hop, directed traversals. Edges are created and removed through the engine's API.

7. Real-time events & streaming

unidb has a durable event queue built in. When you opt a table into event capture, every insert, update, and delete on that table also durably records an event — **inside the same transaction as the change itself**. That inline, transactional capture gives you properties an external message queue cannot:

- **Events can never disagree with your data.** An event is committed with the change that produced it; if that transaction rolls back, its events vanish with it. There is no dual-write gap.
- **No data loss across a crash.** Consumer offsets are stored durably and advanced only when a consumer acknowledges. A crash mid-delivery redelivers rather than dropping — at-least-once delivery is proven, not promised.

- **Kafka-style consumption:** poll from your durable offset, process, then acknowledge. A cleanup pass reclaims events once *every* registered consumer has moved past them — a slow consumer holds its events until it catches up, and cannot be starved.
- **Realtime fan-out.** The REST server exposes the stream as Server-Sent Events (live tail with resume-from-offset), and a companion dispatcher can push events to webhooks (with automatic retry and a dead-letter table) or broadcast rooms — the durable-by-default alternative to a typical realtime layer.

7.1 Delivery performance — flat latency, push-based

Event delivery is backed by its own durable index, so polling cost stays flat no matter how large the event history grows — a table with 300,000 accumulated events polls exactly as fast as one with 10,000 (all in the tens of microseconds). Reclaiming acknowledged events keeps that index itself compact, so the two never work against each other.

New events also **push** a wake-up to waiting subscribers the instant their transaction commits, instead of waiting for the next fixed poll tick — an idle subscriber costs zero CPU and sees a new event in well under a millisecond, rather than up to the old half-second poll interval. Polling remains as an automatic fallback, so nothing changes for existing consumers.

Measured, unidb's event-poll cycle runs in tens of microseconds — flat, at any table size — versus a few milliseconds for the equivalent Postgres `SKIP LOCKED` pattern.

7.2 Offsets, multiple consumers & falling behind

Each named consumer tracks its own durable position in the stream — a single number, its **offset** — advanced only when that consumer explicitly acknowledges what it processed. Polling never moves the offset by itself: you poll to read, then separately acknowledge to say "I've safely handled everything up through here." A crash between the two just redelivers the same events next time — nothing is lost, and duplicates are safe to detect by each event's own sequence number.

Multiple consumers, fully independent

Any number of named consumers can read the same stream at their own pace, each with its own offset — a billing worker three events behind and an analytics job three million events behind coexist without affecting each other. Neither can starve or block the other; there's no shared "head of line."

What happens if a consumer falls behind

Falling behind never slows anything down — polling stays fast regardless of how large the backlog grows (Section 7.1). What it affects is **cleanup**: old events are only reclaimed once *every* registered consumer has acknowledged past them, so one stuck consumer holds the whole table's cleanup hostage — the event history keeps growing until that consumer catches up, or is removed.

That makes lag worth watching, not something that silently breaks anything:

- **Check it directly** — `SELECT * FROM unidb_catalog.subscription_lag` returns, per consumer, how many events behind it is and how old the oldest unprocessed one is.
- **Or watch it continuously** — the same numbers are on `GET /stats` and published as Prometheus gauges (`unidb_subscription_lag_events`, `unidb_subscription_lag_seconds`) — alert at whatever threshold makes sense for that consumer: seconds for a near-real-time pipeline, thousands of events for a batch job.
- **If a consumer is never coming back, remove its registration** — that's what releases the events it was pinning.

Two delivery modes exist depending on whether this durability is what you need: a **named, durable consumer** gets everything above (at-least-once, survives a crash, tracked and alertable); an **ephemeral live tail** (no consumer name — the shape a browser tab watching a table live would use) keeps no server-side offset at all and simply resumes from wherever the connection tells it to on reconnect. Choose durable when losing an event matters; choose ephemeral when you only care what happens next.

7.3 Change-data-capture (CDC) compatibility

Every captured event carries the full **before** and **after** row image (for an INSERT, only `after`; for a DELETE, only `before`; for an UPDATE, both) plus a millisecond timestamp — the same shape a dedicated CDC pipeline (Debezium-style) is built to produce, but generated inline with no extra moving parts. This turns unidb into a self-contained change feed: no separate log-tailing connector to deploy, and nothing to keep in sync with the engine's own storage format.

- **Drop-in wire formats.** `GET /events/subscribe` accepts a `format` parameter — `native`, `debezium`, or `supabase` — so existing Debezium- or Supabase-shaped consumers can point at unidb without rewriting their event handlers.
- **Lag observability out of the box.** Per-consumer lag (how many events behind, and how old the oldest unconsumed event is) is queryable as an ordinary table, exposed on `/stats`, and published as Prometheus metrics — so a consumer falling behind is something you can alert on, not something you discover after the fact.

8. The REST server — full API reference

The optional REST server exposes the whole engine over HTTP with JWT authentication, TLS, realtime streaming, and metrics. It is a thin wrapper over the embedded engine: by default each request runs as one complete `begin → execute → commit` transaction, and many requests execute concurrently against one shared engine. The default embedded library has no network dependencies — the server is a separate, opt-in build.

8.1 Getting started

- **Base URL:** `http://127.0.0.1:8080` by default (configurable). Set a TLS certificate and key to serve HTTPS.
- **Authentication:** every route except `GET /metrics` requires a bearer token: `Authorization: Bearer <jwt>`. Tokens are verified (HS256) against a shared secret; the server does not issue them (plug in your own auth service). The token's `sub` claim is the acting username for per-user privileges.
- **Content type:** JSON in and out, except the raw-row routes (`/rows…`) which carry opaque bytes.
- **Errors:** every non-2xx JSON response has the shape `{ "error": "message", "code": "MACHINE_CODE" }` — see §8.8.

8.2 Running SQL

`POST` `/sql`

Execute one or more `;`-separated SQL statements atomically. If any statement fails, the whole request rolls back.

Body

```
{
  "sql": "INSERT INTO users (email, plan) VALUES ($1, $2)",
  "params": ["ada@example.com", "pro"],   // optional, injection-safe $n binds
  "isolation": "serializable",            // optional: read_committed | repeatable_read |
serializable
  "cursor": false                         // optional: true buffers a large result, returns a
cursor
}
```

Bind values: a JSON string binds as text (coerced to the column type), a number as int/float, a numeric array as a vector.

Response `200 OK` — one result object per statement, in order:

```
{
  "results": [
    { "type": "inserted", "count": 1 }
  ]
}
```

Result shapes include `created_table`, `created_index`, `inserted` / `updated` / `deleted` / `truncated` (with a `count`), `altered_table`, `dropped_table`, and for queries:

```
{
  "type": "rows",
  "columns": ["id", "name", "profile"],
  "rows": [ [1, "alice", { "status": "active" }] ]
}
```

A `JSON` column returns nested JSON; a `DECIMAL` returns a decimal string (e.g. `"9.90"`) and a `TIMESTAMP` a UTC string, so no precision is lost. The same route serves joins, aggregates, `GROUP BY` / `HAVING`, subqueries, `WITH`, `ANALYZE`, and `EXPLAIN [ANALYZE]` — no separate routes.

Cursor mode (`"cursor": true`, one rows-producing statement only) buffers the result server-side and returns `{ "cursor_id": 7, "columns": [...], "row_count": 120000 }` — page it with `GET /sql/cursor/{id}`.

GET `/sql/cursor/{cursor_id}` · **DELETE** `/sql/cursor/{cursor_id}`

Page (or drop) a cursor from a `cursor: true` request.

Query params: `limit` — rows per page (default 1000, max 10 000).

Response `200 OK`:

```
{ "columns": ["id"], "rows": [[1],[2]], "done": false, "remaining": 118000 }
```

The final page reports `"done": true` and drops the cursor. Cursors are bound to the creating principal and expire after 60s idle; `DELETE` drops one early (`204`).

POST `/cypher`

Run a Cypher-subset graph read.

```
// Body
{ "query": "MATCH (a)-[:FOLLOWS]->(b) WHERE a.id = 1 RETURN b.id" }

// Response 200 OK
{ "results": [ { "type": "rows", "columns": ["id"], "rows": [[2],[3]] } ] }
```

8.3 Transactions over HTTP

For work spanning multiple requests, open a **transaction session**. Pass its id on later requests via the `X-Txn-Id` header; those requests run in the transaction and do **not** auto-commit.

POST `/txn/begin`

```
// Body (optional; empty = read_committed)
{ "isolation": "read_committed" | "repeatable_read" | "serializable" }

// Response 201 Created
{ "txn_id": 42, "isolation": "read_committed",
  "idle_timeout_secs": 60, "expires_at": "2026-07-11 12:34:56" }
```

`expires_at` is a sliding idle deadline — each request on the session pushes it out again.

`POST` `/txn/{txn_id}/commit` · `POST` `/txn/{txn_id}/rollback`

```
// Response 200 OK
{ "txn_id": 42, "state": "committed" }   // or "rolled_back"
```

Session rules: one request at a time per session (a second concurrent request gets `409 TXN_BUSY`); a session belongs to the principal that created it (`403 TXN_FORBIDDEN` otherwise); abandoned sessions are auto-aborted after an idle timeout (default 60s) and return `404 TXN_NOT_FOUND`; DDL is not allowed inside a session (`400 DDL_IN_SESSION` — run it as a one-shot request); a failed mutating statement rolls the session back. Sessions do not survive a server restart.

8.4 Rows, graph & indexes

The `/rows…` routes store **opaque bytes** (raw payloads unidb does not interpret) — use `/sql` for typed, columnar data. All accept `X-Txn-Id` to join a session.

Route	Purpose & payload	Response
`POST` `/rows`	Insert one raw row; body is raw bytes.	`201` `{ "row_id": { "page_id": 3, "slot": 0 } }`
`POST` `/rows/batch`	Insert up to 10 000 raw rows atomically; body `{ "rows": ["<base64>", …] }` (32 MiB total).	`201` `{ "row_ids": [ … ] }`
`GET` `/rows/{page_id}/{slot}`	Read a row by id.	`200` raw bytes, or `404 NOT_FOUND`
`PUT` `/rows/{page_id}/{slot}`	Update a row; body is the new raw bytes.	`200` `{ "row_id": { … } }` (id may change)
`DELETE` `/rows/{page_id}/{slot}`	Delete a row.	`204 No Content`
`POST` `/edges`	Create an edge; body `{ "from_id": 1, "to_id": 2, "edge_type": "FOLLOWS", "props": { … } }` (`props` optional).	`201` `{ "row_id": { … } }`
`DELETE` `/edges/{page_id}/{slot}`	Delete an edge; body `{ "from_id": 1 }` (required).	`204 No Content`
`GET` `/edges/from/{from_id}`	List outgoing edges from a vertex.	`200` `{ "edges": [ { "row_id", "to_id", "edge_type", "props" } ] }`
`POST` `/indexes`	Create an index; body `{ "table": "documents", "column": "embedding", "kind": "Hnsw" }` (`kind`: `"Hnsw"` on a vector column or `"FullText"` on a text column; omit `kind` to drop).	`204 No Content`
`GET` `/indexes/{table}/{column}/status`	Report a column's index status.	`200` `{ "status": "Ready" }` or `{ "status": null }`

`GET` `/tables`	List user tables and their columns (cheap, no scan). For full introspection query `information_schema.*` over `/sql`.	`200` array of `{ "name", "columns": [ { "name", "type", "nullable", "index" } ] }`
`POST` `/tables/{table}/bulk`	Bulk-insert NDJSON rows into a table in one transaction — one begin, one prepared `INSERT`, N rows, one commit. Content-Type `application/x-ndjson`, one JSON object per line; the first row's keys set the column order, missing keys become `NULL`. Any error (malformed line, type mismatch, constraint violation) rolls back the whole batch — a partial insert is never left visible.	`200` `{ "inserted": N, "errors": 0, "elapsed_ms": N }`

8.5 Events & realtime

Route	Purpose & payload	Response
`POST` `/tables/{table}/events`	Opt a table into event capture. No body.	`204 No Content`
`GET` `/tables/{table}/events`	Query whether event capture is on for a table.	`200` `{ "enabled": true|false }`
`DELETE` `/tables/{table}/events`	Turn event capture off. Already-captured events are kept until consumed and vacuumed — only future writes stop emitting events. Returns success even if it was already off.	`204 No Content`
`GET` `/events/head`	Return the current highest committed event sequence number without opening a stream — a cheap way to position a new subscriber at "now" instead of replaying full history.	`200` `{ "seq": N }`
`POST` `/events/ack`	Advance a consumer's durable offset; body `{ "consumer": "billing-worker", "up_to_seq": 17 }`.	`204 No Content`
`POST` `/events/vacuum`	Reclaim events acknowledged by *every* consumer. No body.	`200` `{ "reclaimed": 17 }`

`GET` `/events/subscribe` (Server-Sent Events)

Stream new events as SSE frames — pushed the instant a matching transaction commits, with polling as an automatic fallback. Two modes:

- **Durable consumer** (`consumer` set): at-least-once, resumes from that consumer's acked offset; un-acked events are re-yielded until acked.
- **Ephemeral live-tail** (`consumer` omitted): browser-style tail. Resumes past the standard `Last-Event-ID` reconnect header, else `?from_seq=`, else the start.

Query params: `consumer`, `from_seq`, `table` (filter), `limit` (per poll, default 100), `interval_ms` (poll fallback interval, default 500), `format` — `native` (default), `debezium`, or `supabase`.

Response `200 OK`, `text/event-stream` (native format):

```
id: 17
event: insert
data: {"seq":17,"xid":42,"table_name":"orders","op":"insert","ts_ms":1752400000123,
       "before":null,"after":{"id":1,"total":9.99},"payload":{"id":1,"total":9.99}}
```

`before` / `after` are the pre- and post-image of the row (an INSERT has no `before`, a DELETE has no `after`); the flat `payload` field is kept for existing consumers. Passing `?format=debezium` or `?format=supabase` reshapes the same event into that ecosystem's on-wire envelope. Acknowledge separately via `POST /events/ack` — acks are not sent over the stream.

GET `/stats` — subscription lag

Per-consumer lag is included in the standard stats snapshot (and as Prometheus gauges on `/metrics`) — how many unacknowledged events a consumer is behind, and the age in seconds of its oldest unconsumed event. The same numbers are queryable as an ordinary table via `SELECT * FROM unidb_catalog.subscription_lag` over `/sql`.

8.6 Admin, stats, logs & metrics

Route	Purpose & payload	Response
PUT `/tables/{table}/rls`	Attach a row-level-security policy (superuser); body `{ "predicate": "tenant_id = 7" }` (an AND-only `WHERE` subset).	`204`; `400 SQL_PARSE_ERROR` / `403 PERMISSION_DENIED`
POST `/admin/flush`	Force the log durable, then flush all dirty pages (superuser). No body.	`204 No Content`
POST `/checkpoint`	Checkpoint: flush, write a checkpoint record, trim the log. No body.	`204 No Content`
GET `/stats`	Activity + health snapshot (see below).	`200` JSON
GET `/stats/history`	A rolling window (up to 300 points) of timestamped stats snapshots, with commit/WAL rates computed between consecutive points — lets a dashboard prefill its charts on load instead of starting empty. Params: `points` (default 60), `interval_ms`.	`200 { "interval_ms", "points": [ { "t", "commits", …, "commits_per_sec" } ] }`
PUT `/config/slow_query_threshold_ms`	Superuser. Adjust the slow-query logging threshold at runtime, no restart; body `{ "threshold_ms": N }` (`0` disables). Statements over the threshold are logged and appear in `GET /stats`'s `recent_slow_queries`.	`204 No Content`
GET `/logs`	Bounded, cursor-paged tail of structured JSON logs (superuser). Params: `level`, `since`, `until`, `q`, `cursor`, `limit` (≤500).	`200 { "logs": [ … ], "returned", "scanned", "truncated", "next_cursor" }`
GET `/metrics`	Prometheus exposition. **The only route with no auth.**	`200 text/plain`

`GET /stats` is a `pg_stat_*`-style snapshot: commit / abort / checkpoint counts, active transactions, WAL bytes, per-statement-kind latency percentiles, WAL-flush latency, buffer-pool hit ratio, lock waits and deadlocks,

the vacuum-horizon age, per-table page counts *and* per-table cleanup pressure, parallel-worker utilization, autovacuum activity, recent slow queries, open sessions/cursors, and per-consumer event-stream lag. Example fragment:

```
{
  "commits": 42, "aborts": 3, "active_transactions": 0, "wal_bytes": 81920,
  "statement_latency": { "insert": { "count": 50, "p50_us": 32, "p99_us": 256 } },
  "bufferpool": { "hits": 980, "misses": 40, "hit_ratio": 0.9607 },
  "locks": { "waits": 0, "deadlocks": 0 },
  "horizon_age_secs": 0.0,
  "tables": [ { "name": "t", "pages": 3, "dead_tuple_estimate": 2, "live_tuple_estimate": 118
} ]
}
```

8.7 Replication

Route	Purpose
`POST` `/replication/slots`	Create a slot; body `{ "name": "…", "sync": false }` → `201 { "name", "restart_lsn", "kind" }`.
`GET` `/replication/slots`	List slots → `{ "slots": [ … ] }`.
`DELETE` `/replication/slots/{name}`	Drop a slot → `204`.
`POST` `/replication/slots/{name}/advance`	Confirm applied up to an LSN; body `{ "lsn": <n> }` → `204`.
`GET` `/replication/stream?from_lsn={n}`	Ship log records after `from_lsn` as a byte stream; the primary's tail LSN is in the `x-unidb-tail-lsn` response header.

8.8 Object storage

Seven routes expose the engine's built-in object store — large files tiered to S3/MinIO-compatible storage, with their metadata living in ordinary catalog tables (see §6.2 for the embedding-search-over-files pattern). All require a bearer token. If object storage isn't configured on this server, every route below returns `503 { "error": "…", "code": "STORAGE_NOT_AVAILABLE" }` rather than failing in a less predictable way.

Route	Purpose & payload	Response
`GET` `/storage/buckets`	List buckets.	`200 { "buckets": [ { "name", "created_by", "created_at_ms" } ] }`
`POST` `/storage/buckets`	Create a bucket; body `{ "name": "…" }`.	`201`
`DELETE` `/storage/buckets/{name}`	Delete an empty bucket.	`204`; `409` if the bucket still has objects
`GET` `/storage/{bucket}/objects`	List objects, optionally scoped with `?prefix=` and folded into virtual folders with `&delimiter=` (S3-style listing).	`200 { "objects": [ { "object_key", "size", "etag", "content_type", "status", "tier" } ], "prefixes": [ … ] }`

PUT `/storage/{bucket}/objects/{*key}`	Upload an object. Below the inline threshold (default 1 MiB) the bytes are stored directly, in one transaction; at or above it, the response hands back a presigned URL and the client uploads the bytes straight to storage.	`201 { "tier": "inline", "size", "etag" }` or `200 { "presigned_put_url", "storage_key" }`
DELETE `/storage/{bucket}/objects/{*key}`	Delete an object.	`204`; `404` if not found
GET `/storage/{bucket}/presign/{*key}`	Get a time-limited direct-download URL, so a browser or client can fetch the object without holding an app credential.	`200 { "presigned_get_url": "…" }`

8.9 Error codes

Every error carries a stable machine-readable `code`:

HTTP	Code	Meaning
404	`TABLE_NOT_FOUND` / `COLUMN_NOT_FOUND` / `NOT_FOUND`	Referenced table/column doesn't exist; row has no visible version
404	`TXN_NOT_FOUND` / `CURSOR_NOT_FOUND`	Unknown/finished/expired session or cursor id
409	`TABLE_ALREADY_EXISTS`	`CREATE TABLE` on an existing name
409	`WRITE_CONFLICT` / `SERIALIZATION_FAILURE` / `DEADLOCK`	Concurrency conflict — retry the transaction
409	`UNIQUE_VIOLATION`	Duplicate `UNIQUE` / `PRIMARY KEY` value
409	`TXN_BUSY`	Second concurrent request on one session
408	`QUERY_TIMEOUT` / `QUERY_CANCELLED`	Per-query time budget or cancellation
403	`PERMISSION_DENIED` / `TXN_FORBIDDEN` / `CURSOR_FORBIDDEN`	Missing privilege / not your session or cursor
400	`SQL_PARSE_ERROR` / `SQL_PLAN_ERROR` / `SQL_UNSUPPORTED`	Malformed or unsupported SQL
400	`NOT_NULL_VIOLATION` / `CHECK_VIOLATION` / `FOREIGN_KEY_VIOLATION`	Constraint failed
400	`DDL_IN_SESSION` / `ISOLATION_IN_SESSION` / `CURSOR_NOT_ROWS`	Operation not allowed in this context
400	`EMPTY_BATCH` / `BAD_BASE64` / `BATCH_TOO_LARGE`	Invalid batch-insert payload
401	`UNAUTHORIZED`	Missing/invalid/expired token
503	`DURABILITY_FAILURE`	A disk flush failed; the engine must be restarted (durability can no longer be guaranteed)
503	`STORAGE_NOT_AVAILABLE`	Object storage isn't configured on this server
409	`BUCKET_NOT_EMPTY`	Bucket delete attempted while it still holds objects
500	`INTERNAL_ERROR`	I/O, checksum, or corruption a well-formed request should never trigger

9. Operations & high availability

- **Segmented write-ahead log.** The log is kept as fixed-size segments; consumed segments are deleted whole. Checkpoints run automatically on a time or size trigger (at a quiescent point) to keep the log bounded and recovery fast.
- **Replication & read replicas.** A replica seeds from a base snapshot and applies shipped log incrementally. The primary ships only data it has made durable, so a replica is always a consistent prefix of the primary — it can never show a change the primary might lose. On failover, a replica is promoted to read-write, with an optional synchronous-commit gate for zero-loss failover.
- **Backups & point-in-time restore.** Take a base backup plus archived log and restore to a chosen point — either **by wall-clock time** ("restore to 2:14 pm yesterday") or by exact log position for byte-precise control. A restore-to-time request resolves the nearest safe commit at or before that moment, so you never overshoot into an uncommitted change. Measured on a sample database: base backup 7 ms, restore 72 ms, failover 26 ms.
- **Logical replication.** Beyond byte-for-byte physical replicas, unidb can replicate at the row level into a different target database — translating each captured change back into SQL and applying it there. This is the foundation for migrating into unidb from elsewhere, keeping a reporting copy in sync, or fanning changes out to a differently-shaped downstream schema; it reuses the same durable, at-least-once event delivery as the realtime stream (Section 7), so a restarted replication process resumes exactly where it left off rather than re-copying or dropping data.
- **Autovacuum.** A background worker tracks dead-row buildup **per table** and reclaims space table by table, so one churn-heavy table doesn't force a scan of every other table. A cost-based throttle caps how much background I/O a vacuum pass is allowed to use, so cleanup never competes hard enough with live traffic to matter. It scrubs every index before space is reused, and under heavy churn keeps a table compact automatically (bounded at ~35 pages vs ~82 unvacuumed in one test) while keeping point reads fast.
- **Security & observability.** Users, roles and privileges; verified JWT identities; native TLS; an append-only audit log; lock-free health metrics over `/stats` and `/metrics`; and structured JSON logs with a request id that correlates a single request across the app, slow-query, and audit logs (ready to ship to CloudWatch / Datadog / Loki). An operations runbook ships with the engine.

10. Performance — measured against Postgres

All numbers below are from release builds on Apple Silicon at real flush-to-disk durability, measured with the project's benchmark harness against Postgres (with pgvector, at matched durability) and, for the headline, against the full four-system stack unidb replaces. unidb reports honest wins *and* the places it still trails.

10.1 The multi-model advantage, quantified

Measurement	Result
Save a row + its embedding + a graph edge + an event, one durable commit, vs. the same across Postgres + a vector store + a graph table + an outbox (four commits)	**3.61× faster** (250 vs 69 txns/s) at real durability
Crash consistency across those four writes	**Unconditional win:** 0 orphans vs. a torn record
Adding each extra model to the same commit	Four models cost about the **same as one** (they share a single flush)

10.2 Where unidb wins on single-model work

Workload	Result vs. Postgres
`SELECT COUNT(*)`	**2.81× faster**
Filtered scans (parallel workers, on by default)	**6.4–6.6× faster**; full scan up to ~35.7M rows/s at 1M rows
Event polling (vs Postgres `SKIP LOCKED`)	Tens of µs, **flat regardless of table size**, vs ~2.6–3.1 ms
Embedded point reads	~5× faster (~1.09 µs)
Graph adjacency (hot vertex)	Parity (~94 µs vs ~98 µs at 1k edges)

10.3 What made it fast

Improvement	What it does	Measured effect
Group commit	Many transactions share one durability flush; read-only commits skip it	Multi-model commit 33.1 → 4.3 ms (~7.7×); server 135 → 4,780 requests/s at 50 clients; point read 3.05 ms → ~1.09 µs
Concurrent writers	Thread-safe engine, fine-grained latches, real lock manager	Raw writes scale 3.68× at 8 writers; indexed inserts +25%
Parallel scans (default on)	Scan work partitioned across CPU cores over the shared memory map	COUNT 3.82×, filtered scans up to 6.6×
Partial-decode reads	Decode only the columns a query needs; count from headers alone	`COUNT(*)` 2.81× faster than Postgres
Durable free-space map	Per-table space tracking; instant open	Insert cost flat ~17–28 µs/row into the millions (was degrading, then failing)
Batched graph resolution	Fetch a vertex's edges grouped by page (~128 edges/page)	~9× over naive; parity with Postgres
Durable vector index	Similarity index kept on disk, never recomputed on open	recall@10 = 1.000; open instant (vs 30 s in-memory rebuild)

Indexed event polling	A durable index on event order replaces a linear scan; commits push a wake-up instead of waiting for the next poll	Poll latency flat (~30 µs) from 10k to 300k+ queued events; idle-subscriber delivery drops from up to 500 ms to sub-millisecond

10.4 Scale

Bulk insert holds ~31k rows/s flat to 2M rows; a full-table scan runs at ~17.5M rows/s at 1–2M rows; the engine handles databases of ten million rows and beyond. The confirmed target is a strong single node plus read replicas — hundreds of GB with high read throughput.

11. Configuration & performance tuning

unidb ships with defaults chosen to be **safe for a first run**, not necessarily the fastest possible for every workload. Every setting below is an environment variable read once at engine (or server) start-up; the embedded crate also exposes most of them as explicit constructor/setter calls for programmatic control without touching the environment. Except where noted, the same variable governs both the embedded crate and the REST server, since the server is a thin wrapper over `Engine::open`.

> **The buffer pool is not a data cache — read this before tuning it.** unidb stores pages in a memory-mapped file, so page bytes already live in the OS page cache "for free," the same way they would for any other file the OS has read. The buffer pool (`UNIDB_BUFFER_POOL_PAGES`) only tracks **pin and dirty-tracking metadata** per page (~24 bytes/frame) — not a Postgres-style `shared_buffers` RAM cache of page contents. Sizing it like one (e.g. a large fraction of RAM) wastes memory on frame-table bookkeeping for no caching benefit; sizing it too small for your working set is the real hazard (see 11.1).

11.1 Memory & storage layout

Setting	Default	Purpose	Performance impact / when to override
`UNIDB_BUFFER_POOL_PAGES`	65,536 frames (~512 MiB pin/dirty-tracking ceiling)	Caps how many pages may be simultaneously pinned/dirty in the buffer pool.	Too small for a workload's concurrently-dirtied working set forces a synchronous `wal.sync()` on every write once the pool fills (`BufferPoolFull`) — measured on a demo seed this collapsed insert throughput from a flat ~25k rows/s to ~1.2–1.7k rows/s (93 checkpoints for 211 commits) even though nothing else was wrong. Raise it for large bulk loads across many tables — e.g. 1,000,000 frames costs ~24 MiB of frame-table memory and ~530 µs extra per `Engine::open()` , negligible against the throughput it restores. Do not raise it expecting a page-data cache effect (see the callout above) — its cost is a bigger frame table, not more cached bytes.
`UNIDB_PAGE_SIZE`	`0` → engine default (8 KiB)	Page size baked into the control file at first database creation (locked decision D8).	**Fixed forever once any file exists** — only takes effect on a brand-new data directory's first `Engine::open` ; there is no way to change it later short of dump/reload into a new database. A larger page amortizes the page header over more rows per fetch (fewer page reads for large sequential scans) at the cost of a bigger full-page WAL image the first time a page is dirtied after each checkpoint — leave it at the default unless you have measured a specific large-row or large-scan workload that benefits.

11.2 Write durability & the WAL

Setting	Default	Purpose	Performance impact / when to override

`UNIDB_AUTO_CHECKPOINT`	on	Master switch for automatic checkpointing, which bounds WAL growth and recovery replay time.	Disabling it lets the WAL grow unbounded until a manual checkpoint — never disable in production; it exists for tests that want deterministic manual control over checkpoint timing.
`UNIDB_CHECKPOINT_TIMEOUT_SECS`	60	Checkpoint at least this often once the engine is quiescent (no open transaction).	Lowering it bounds worst-case recovery time and WAL disk usage more tightly at the cost of more frequent checkpoint I/O (and more full-page images re-logged after each one); raising it trades a longer potential recovery replay for less checkpoint overhead on a write-heavy but low-idle-time workload.
`UNIDB_MAX_WAL_SIZE_BYTES`	64 MiB	Checkpoint once the WAL since the last checkpoint reaches this many bytes — the size-based trigger alongside the timeout above.	Lower it on a disk-constrained host to cap WAL size tightly; raise it on a write-heavy host to amortize checkpoint cost over more writes, accepting a larger WAL and longer recovery replay after a crash.
`UNIDB_WAL_SEGMENT_BYTES`	16 MiB	Size of each rotated WAL segment file; consumed segments are deleted whole once no longer needed (including by replication slots).	Mostly an operational knob, not a throughput one: smaller segments free disk space in finer-grained increments (useful when a replication slot or slow consumer is holding segments back) at the cost of more file handles/rotations; larger segments reduce rotation overhead on a high-throughput write stream.

11.3 Query execution & concurrency

Setting	Default	Purpose	Performance impact / when to override
`UNIDB_PARALLEL_SCAN`	on	Master switch for partitioning a table scan's pages across worker threads instead of scanning serially.	Default-on because a governed global worker budget (below) prevents oversubscription. Measured 3.82× on `COUNT(*)` and up to 6.4–6.6× on filtered scans. Turn off only as a rollback if parallel scans ever regress a specific query (e.g. a workload dominated by many small concurrent scans where thread hand-off cost outweighs the win).
`UNIDB_PARALLEL_MIN_PAGES`	64 pages	Minimum table size before a scan is considered worth parallelizing at all.	Raise it if small-table scans show thread-spin-up overhead in your profile; lower it to parallelize smaller tables too (rarely a net win — the fixed cost of coordinating workers dominates below a few dozen pages).
`UNIDB_PARALLEL_MAX_WORKERS`	0 → one worker per CPU core	Per-query cap on how many workers a single scan may use.	Lower it to leave headroom for other concurrent work on the same cores when one query shouldn't be allowed to claim the whole machine; leave at the default (all cores) for a dedicated batch/analytics workload.

`UNIDB_PARALLEL_MAX_TOTAL_WORKERS`	= CPU core count	The **global** ceiling on live parallel-scan worker threads across *all* concurrent queries at once — extra scans degrade to serial instead of oversubscribing the machine.	This is the actual oversubscription guard: lower it on a host shared with other services (databases, app servers) so scans don't starve them; raise it on a dedicated many-core box running mostly scan-heavy analytical queries. The `/stats` / `/metrics` worker counters (§8.6) show how often the global budget was actually exhausted (`serial_fallbacks`) — a non-zero, growing count is the signal to raise this.
`UNIDB_CONCURRENT_SQL_WRITES`	on	Lets non-DDL DML statements from multiple writers take a *shared* catalog lock and run concurrently, instead of serializing all SQL writes behind one lock.	Default-on after passing a 28-cell concurrency correctness matrix both ways; measured 8-writer indexed `INSERT` recovering from ~811 to ~1016 commits/s toward the ~1275 commits/s unindexed floor. Set to `0` only as an emergency rollback if a concurrency-related correctness issue is ever suspected in production — it reproduces the fully serialized path exactly, at a real throughput cost under concurrent writers.
`UNIDB_SORT_MEM_ROWS`	1,000,000 rows	In-memory row budget for an `ORDER BY` before it spills to an external merge sort on disk.	Raise it on a memory-rich host to keep larger sorts fully in memory (avoids spill I/O); lower it to bound peak memory per query on a memory-constrained or highly concurrent host. Overridable per-query via the engine's `work_mem` query-limits API without touching the process-wide default.
`UNIDB_HASH_JOIN_MEM_ROWS`	1,000,000 rows	In-memory row budget for a hash join's build side before it spills to disk (Grace hash join).	Same trade-off as `UNIDB_SORT_MEM_ROWS` above, applied to joins: raise for faster large joins if RAM allows, lower to bound memory, or override per-query via `work_mem` .

11.4 Vacuum & space reclamation

Setting	Default	Purpose	Performance impact / when to override
`UNIDB_AUTOVACUUM_ENABLED`	on	Master switch for the background worker that reclaims dead-row space automatically, per table.	Disabling it stops automatic cleanup entirely — dead rows accumulate until a manual vacuum, growing the table on disk and slowing scans over time. Leave on in production; disable only for a benchmark that wants to isolate a code path from background vacuum noise.
`UNIDB_AUTOVACUUM_THRESHOLD`	50 dead rows	Minimum dead-tuple count before a vacuum is even considered for a table (Postgres's own default).	Lower it to reclaim space more eagerly on small, high-churn tables; raise it to avoid vacuuming very small tables at all.

`UNIDB_AUTOVACUUM_SCALE_FACTOR`	0.2	Fraction of a table's live-row estimate added to the threshold above, so larger tables tolerate proportionally more churn before vacuuming.	Lower it to vacuum large, high-churn tables more aggressively (more background I/O, less bloat); raise it to let large tables tolerate more dead space before paying for a vacuum pass.
`UNIDB_AUTOVACUUM_NAPTIME_SECS`	60	How often the background launcher wakes to re-check the trigger condition above, per table.	Lower it for faster reaction to sudden churn spikes at the cost of more frequent (cheap) wake-ups; raise it to reduce background wake-up overhead on an otherwise idle system.
`UNIDB_VACUUM_COST_ENABLED`	on	Master switch for a cost-budget throttle (mirroring Postgres's `vacuum_cost_*` settings) that caps how much I/O a vacuum pass may consume before napping.	Keeps vacuum from competing hard enough with live traffic to matter — a churn test held a table to ~35 pages vs ~82 unvacuumed while keeping point reads fast. Disabling it lets a vacuum pass run flat-out (finishes reclaiming space sooner) but can visibly compete with concurrent read/write I/O on a busy server — only disable for an offline/maintenance-window vacuum where nothing else is running.

11.5 REST server timeouts (server deployments only)

Setting	Default	Purpose	Performance impact / when to override
`UNIDB_REQUEST_TIMEOUT_SECS`	120	Global HTTP request timeout applied to every route.	Sized to comfortably cover a 100k-row bulk-load payload by default. Lower it to fail fast on a latency-sensitive API surface; raise it (or set to `0` to disable entirely) for larger bulk-load jobs or local development tooling where an open-ended request is expected.
`UNIDB_TXN_IDLE_TIMEOUT_SECS`	60	Idle deadline for an HTTP transaction session before the reaper auto-aborts it.	An abandoned open transaction holds locks and pins the vacuum horizon (blocking space reclamation), so lowering it recovers those resources faster from misbehaving/disconnected clients; raising it accommodates a legitimately slow interactive client at the cost of holding locks/horizon longer per idle session.
`UNIDB_CURSOR_IDLE_TIMEOUT_SECS`	60	Idle deadline for a `POST /sql` result cursor before it is reclaimed.	Same trade-off as the transaction timeout above, scoped to server-side cursor memory instead of locks: lower to reclaim idle cursor state sooner, raise for slow paginating clients.
`UNIDB_SLOW_QUERY_MS`	unset (disabled)	Threshold above which a query is logged to the slow-query log with its correlation id.	Not itself a performance lever, but the fastest way to *find* the next tuning target: set it (e.g. to 100) in any environment where query latency matters, and let the resulting slow-query log point at which of the settings above actually needs changing.

12. Correctness & testing

unidb's safety claims are backed by tests, not assertions of good intent:

- **Crash testing.** A dedicated harness kills the engine at dozens of precise points — after a log write but before a flush, mid-checkpoint, mid-recovery, during log trimming, mid-vacuum, on a torn page, on a simulated disk-flush failure, and after total data-file loss — then reopens and asserts the database contains exactly the committed transactions. A property test drives random transaction sequences with random crash points under both durability settings.
- **Concurrency testing.** A production-shaped concurrency matrix checks exact visible-row sets, no duplicate rows in any snapshot, agreement between index and scan paths, and repeatable reads — under CPU contention and with hang deadlines.
- **Differential testing.** Join, aggregate, and subquery results are compared against SQLite on shared data.
- **Every change is gated.** Formatting, linting, the full test suite, the crash harness, and recorded benchmarks must all pass before any change lands.

The engine is rigorously tested and crash-safe by design, but it is still early-stage software — it has not accumulated the decades of production hardening that Postgres has. Section 13 is an honest account of where it stands.

13. Known limitations, bugs & roadmap priority

unidb keeps an honest, public account of its gaps — both missing features and known bugs — so nothing below is a surprise in production. Every item carries a **priority** (how much it matters to close) and a status: actively **Tracked**, or explicitly **Planned** for a future phase.

13.1 Known bugs (open, tracked)

This is the complete list of currently open, user-visible correctness or behavior bugs. It is short by design: every other correctness bug uncovered during development — including a subtle transaction-rollback ordering issue found under heavy concurrent load — was root-caused, fixed, and is now locked in place by a permanent automated regression test (Section 12). Nothing below causes silent data loss or a wrong answer; each is a query that is rejected when it should be accepted.

Bug	What happens	Priority
Inconsistent `IS NULL` / `IS NOT NULL` filter support	`WHERE column IS NULL` works on a query that already sorts, joins, or aggregates, but the identical filter is rejected as unsupported on a plain filtered lookup — and, notably, when combined with a vector similarity search (`NEAR(...) AND column IS NULL`). The same predicate silently behaves differently depending on unrelated parts of the query, which is a genuine surprise for anyone hitting it.	High — blocks a common "similarity search + filter" pattern

13.2 Known limitations & feature gaps

Area	Limitation	Priority	Status
Transactional DDL	A `CREATE TABLE` in a transaction that later aborts is not rolled back through crash recovery (per-request rollback does apply today)	High	Tracked
Serializable isolation	Row-granularity only — no predicate locks, so no phantom protection; sound but occasionally over-conservative	High	Tracked
Auth platform	The server verifies tokens but does not issue them — no built-in sign-up/login/OAuth	High	Planned — next phase
SQL surface	No window functions, recursive CTEs, FULL OUTER / NATURAL joins, views, triggers, or stored procedures; single-column indexes only	Medium	Planned — next phase
Event/CDC row-level security	The event and change-data-capture stream is not yet filtered by a table's row-level-security policy — a subscriber sees every row's change on a table it has access to poll, not only the rows its policy would allow through ordinary SQL	Medium	Tracked
Vacuum	Per-table cleanup and a background I/O throttle are in place; still no whole-table compaction (cross-page defragmentation) to shrink a table back down after heavy churn	Low	Tracked
Logical replication	Row-level replication into another database requires the target schema to already exist (no automatic schema creation) and has no column-remapping between source and target	Low	Tracked
Updates	An UPDATE writes a new row version (more write volume than Postgres's in-place update path); measured ~0.34× of Postgres on bulk updates	Low	Planned — next phase
Encryption at rest	Not yet available	Low	Planned — next phase

Graph	Cypher is single-hop, read-only; no reverse-direction traversal yet	Low	Tracked
Maturity	Rigorous and crash-tested, but early-stage — not yet battle-hardened over decades like Postgres	Low	Ongoing

14. Where unidb is headed

The roadmap closes the gaps that block production adoption while protecting the one thing that makes unidb different — multi-model atomicity. It is framed against the two systems users compare unidb to: **Postgres** (as an engine) and **Supabase** (as a backend platform).

14.1 Toward Postgres-level engine completeness

Theme	What's coming
Transactional completeness	Transaction-scoped, crash-safe DDL; savepoints and nested rollback; full serializable isolation with phantom protection; complete foreign-key actions (`ON DELETE / UPDATE CASCADE` , etc.)
SQL surface	Views and materialized views; window functions; triggers and stored procedures; recursive CTEs and remaining join types; richer types (arrays, ENUM, INTERVAL, ranges); multi-column, partial, and expression indexes
Performance parity	In-place (HOT-style) updates to close the update gap; more query work pushed onto parallel workers
Ecosystem & operations	A PostgreSQL wire-protocol listener so existing drivers, ORMs, and BI tools connect unchanged; encryption at rest

14.2 Toward a backend-in-a-box (Supabase-style platform)

Several of these have already shipped; others are planned.

Capability	Today	Direction
Realtime	Durable SSE stream with resume + webhook / broadcast fan-out	**Shipped** — durable-by-default, a genuine advantage over realtime layers that aren't
Observability	Lock-free per-chokepoint metrics over `/stats` and `/metrics`	**Shipped**
Logs	Structured JSON logs with request-id correlation and a bounded `/logs` tail	**Shipped**
Auth platform	Token verification, users, roles, per-table grants	**Planned** — a full auth service (sign-up / login, OAuth, magic links, MFA) in front of the existing core
Client SDKs	REST + Rust attach client	**Planned** — TypeScript SDK first, then Python, generated from the REST contract
Auto REST richness	Hand-built routes; SQL over HTTP	**Planned** — PostgREST-style filtering / ordering / embedding, OpenAPI generation
Object storage	Transactional large objects inside the engine (multi-GB), tiered to S3/MinIO-compatible storage with presigned upload/download and automatic reconciliation	**Shipped**

Change-data-capture	Before/after row images on every event, Debezium- and Supabase-compatible wire formats, and per-consumer lag observability (queryable, on `/stats`, and as Prometheus metrics)	**Shipped** — drop-in for teams already standardized on a Debezium- or Supabase-shaped consumer
Time-based point-in-time restore	Restore to a wall-clock time, not just a log position; row-level logical replication into another database	**Shipped**
Dashboard / studio	—	**Planned** — a web console: schema browser, SQL editor with EXPLAIN, metrics, RLS/role editor
Migrations tooling	SQL DDL + docs	**Planned** — versioned migration files and a CLI once transactional DDL lands
Edge functions / hosting	—	**Not planned** — no cloud control plane by design; run unidb embedded inside your own function runtime

14.3 Deliberately parked

- **Columnar / analytical engine** — the opposite axis to the multi-model transactional thesis; gated on real analytics demand.
- **Distributed / sharded multi-writer** — reverses the single-primary design and strains cross-model atomicity; the confirmed target remains a strong single node plus read replicas.

15. Glossary of terms & abbreviations

Plain-language definitions for every abbreviation used in this guide, in the sense it is used here — alphabetical, appendix-style.

Term	Meaning
ACID	Atomicity, Consistency, Isolation, Durability — the four guarantees a transaction gets: all its changes happen together or not at all, the data stays valid, concurrent transactions don't corrupt each other's view, and a commit survives a crash.
ANN	Approximate Nearest Neighbor — the general family of techniques vector similarity search belongs to. unidb's index re-ranks candidates exactly before returning them, so results carry no approximation error despite the family name.
API	Application Programming Interface — the programmatic surface (embedded library calls, or REST routes) an application uses to talk to unidb.
BI	Business Intelligence — reporting and dashboarding tools that connect to a database over a standard driver.
CDC	Change Data Capture — a stream of before/after row images describing every insert, update, and delete, meant for downstream consumers to replay or react to.
CPU	Central Processing Unit — a processor core; parallel table scans split work across CPU cores.
CTE	Common Table Expression — a named subquery introduced with `WITH`, reusable elsewhere in the same SQL statement.
DDL	Data Definition Language — schema-changing statements (`CREATE`, `ALTER`, `DROP`, `TRUNCATE`), as distinct from statements that only read or write rows.
GB / MB / KiB / MiB	Gigabyte / Megabyte / Kibibyte / Mebibyte — units of data size. The "i" (kibi-, mebi-) units are the exact binary sizes (multiples of 1,024) unidb uses internally for page and buffer-pool sizing.
HTTP / HTTPS	Hypertext Transfer (Secure) Protocol — the protocol the REST server speaks; HTTPS is HTTP encrypted with TLS.
I/O	Input/Output — reads and writes to disk. Vacuum's cost-based throttle bounds how much background I/O a cleanup pass is allowed to use.
JSON	JavaScript Object Notation — the data format the REST API sends and receives, and the format stored by the `JSON` column type.
JWT	JSON Web Token — the signed bearer-token format used to authenticate REST requests.
LSN	Log Sequence Number — a strictly increasing position in the write-ahead log. Log-position-based point-in-time restore targets an LSN, as opposed to a wall-clock time.
MFA	Multi-Factor Authentication — signing in with more than a password; a planned addition to the auth platform.
MVCC	Multi-Version Concurrency Control — unidb's concurrency model: an update creates a new row version rather than overwriting in place, so readers and writers never block each other.
OAuth	Open Authorization — a standard for delegated sign-in (e.g. "Sign in with Google"); planned for the auth platform.
ORM	Object-Relational Mapper — an application-side library that maps database rows to objects in your programming language.
REST	Representational State Transfer — the architectural style of unidb's optional HTTP API.

RLS	Row-Level Security — a policy attached to a table that automatically filters every query down to the rows a given caller is allowed to see.
SQL	Structured Query Language — the query language unidb's relational layer speaks.
SSE	Server-Sent Events — a one-way HTTP streaming format used for the realtime event subscription.
TLS	Transport Layer Security — the encryption protocol behind HTTPS; native support ships with the REST server.
UUID	Universally Unique Identifier — a 128-bit identifier type that can be generated independently and still be, in practice, globally unique.
UTC	Coordinated Universal Time — the timezone unidb stores and returns `TIMESTAMP` values in.
WAL	Write-Ahead Log — the durability log every change is recorded to before it is applied to the data file; the foundation of crash recovery (Section 3).

---

unidb — Engine Architecture & Product Guide. This is a distilled reference; where it disagrees with the engine's canonical source docs, those win. Editing and regeneration instructions (and the source-material provenance) are in `unidb_engine_architecture_context.md`, alongside this file.